



复习课（六） （代码生成与寄存器分配）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



代码生成器中的问题



■ 指令选择

- 代码生成器需把IR程序映射成为可以在目标机上运行的代码序列。
- 完成映射的复杂性由如下因素决定：
 - 1. IR层次
 - 2. 指令集体系结构本身特性
 - 3. 要达到代码质量
- 如每个形如 $x=y+z$ 的三地址语句，可被翻译为如下
 - LD R0, y
 - ADD R0, R0, z
 - ST x, R0
- 大多数机器上，一个给定的IR程序可用很多种不同的代码序列来实现
- 如目标机有一个“加一”指令，则 $a=a+1$ 可用一个指令INC a实现



代码生成器中的问题



■ 指令选择

$a = a + 1$

```
LD R0, a      // R0 = a
ADD R0, R0, #1 // R0 = R0 + 1
ST a, R0      // a = R0
```



INC a

得考虑
指令代价



八、简答题（共1题，共10分）

假设有两个可用的寄存器R1和R2，使用简单代码生成算法。

为如下三地址代码生成机器码。

$t = a * b$
 $u = x + t$
 $x = u$

1. 给出每条三地址代码生成机器码后的寄存器描述符和内存描述符（请在表格左边空白部分写出三地址代码，并在表格中补充写入对应的描述符内容）。
2. 结束时将寄存器中的变量写入内存中。

附机器代码格式：

和数指令：LD R, var // 将内存空间中变量var的值存入寄存器R中。

存数指令：ST var, R // 将寄存器R中的值存入变量var的内存空间中。

双目运算指令：OP dst, src1, src2 // 使用OP对寄存器src1和src2中的值进行计算，结果存入寄存器dst中（OP包括ADD, SUB, MUL, DIV）。

初始状态：

R1	R2	x	a	b	t	u
		x	a	b	t	u

(1) $t = a * b$

R1	R2	x	a	b	t	u
		x	a	b	R1	u

(2) $u = x + t$

R1	R2	x	a	b	t	u
t	u	x	a	b	R1	R2

(3) $x = u$

R1	R2	x	a	b	t	u
t			a	b	R1	

(4)

R1	R2	x	a	b	t	u
t	u, x	x, R2	a	b	t, R1	u, R2



代码生成器中的问题



■ 指令选择

$x = y + z$

```
LD R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z  // R0 = R0 + z (add z to R0)
ST x, R0      // x = R0      (store R0 into x)
```

$a = b + c$
 $d = a + e$

```
LD R0, b      // R0 = b
ADD R0, R0, c  // R0 = R0 + c
ST a, R0      // a = R0
LD R0, a      // R0 = a
ADD R0, R0, e  // R0 = R0 + e
ST d, R0      // d = R0
```

如果变量a
不再被使用
了呢？

冗余的指令

思考，是不是一定冗余？



代码生成器中的问题



■ 目标语言

■ 三地址机器的模型

- 寄存器的使用常被分解为两个子问题：
 - 1. 寄存器的分配
 - 2. 寄存器的指派
- 我们的目标计算机是一个三地址机器的模型
- 具有加载和保存操作，计算操作，跳转操作，条件跳转
- 这个计算机的内存按字节寻址，具有n个通用寄存器R0, ..., Rn-1
- 一个完成的汇编语言有几十到上百个指令
- 我们使用一个有限的指令集合
- 假设所有运算分量都是整数
- 大部分指令含一个运算符，然后是一个目标地址，最后是一个源运算分量列表
- 指令之前可能有一个标号



- 三地址机器的模型
- 指令集
 - 加载: LD dst, addr
 - 把地址addr中的内容加载到dst所指寄存器
 - 保存: ST x, r
 - 把寄存器r中的内容保存到x中
 - 计算: OP dst, src1, src2
 - 把src1和src2中的值运算后将结果存放到dst中
 - 无条件跳转: BR L
 - 控制流转向标号L的指令
 - 条件跳转: Bcond r, L
 - 对r中的值进行测试, 如果为真则转向L



- 如何将IR中的名字转换成为目标代码中的地址
 - 不同的区域中的名字采用不同的寻址方式
- 如何为过程调用和返回生成代码
 - 静态分配
 - 栈式分配



- 划分基本块的算法
 - 输入: 三地址指令序列
 - 输出: 基本块的列表
 - 方法:
 - 确定基本块的首指令:
 - 第一个三地址指令
 - 任意一个条件或无条件转移指令的目标指令
 - 紧跟在一个条件/无条件转移指令之后的指令
 - 确定基本块:
 - 每个首指令对应于一个基本块: 从首指令开始到下一个首指令



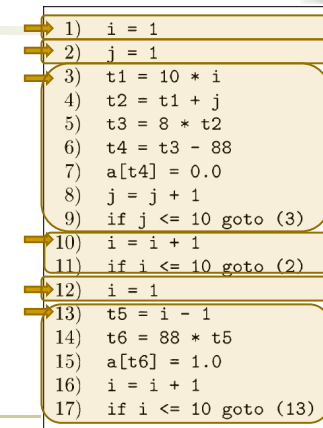
- 程序和指令的代价
 - 不同的目的有不同的度量
 - 最短编译时间、目标程序大小、运行时间、能耗
 - 假设: 每个指令有固定的代价, 设定为1加上运算分量寻址模式的代价
 - LD R0, R1; 代价为1
 - LD R0, M; 代价是2
 - LD R1, *100(R2); 代价为2
 - 为给定源程序生成最优目标程序
 - 不可判定: 不可判定问题是可计算性理论和计算复杂性理论中定义的一类决定性问题, 此类问题无法总是用单一算法得出正确的是/否的答案。



- 中间代码的流图表示法
 - 中间代码划分成为基本块(basic block)
 - 控制流只能从第一个指令进入
 - 除基本块的最后一个指令外, 控制流不会跳转/停机
 - 结点: 基本块
 - 边: 指明了基本块的执行顺序
- 流图可以作为优化的基础
 - 指出了基本块之间的控制流
 - 可以根据流图了解到一个值是否会被使用等信息



- 基本块划分的例子
 - 第一个指令
 - 1
 - 跳转指令的目标
 - 3、2、13
 - 跳转指令的下一条指令
 - 10、12
 - 基本块:
 - 1-1; 2-2; 3-9; 10-11;
 - 12-12; 13-17





基本块和流图

■ 练习

- (1) $b = 1$
- (2) $n = 10$
- (3) $d = l + n$
- (4) $c = 2$
- (5) $t1 = 4 * a$
- (6) $t2 = l + n$
- (7) $c = c + b$
- (8) if $c < n$ goto (6)
- (9) $a = a + b$
- (10) if $a < n$ goto (4)
- (11) $c = a - b$



基本块和流图

■ 后续使用信息

- 变量值的使用 (use) 与活跃 (live)
 - 假设三地址语句 i 给 x 赋了一个值, 如果在后续的另一语句 j 的一个运算分量为 x , 且从 i 开始有一条没有对 x 进行赋值的路径能够到达 j , 那么 j 就使用了 i 处计算得到的 x 的值
 - 我们可以说称 x 在语句 i 后的程序点上 **活跃**
 - 即在程序执行完语句 i 的时刻, x 中存放的值将被后面的语句使用
 - 不活跃是指变量中存放的值不会被使用, 而不是变量不会被使用



基本块和流图

■ 例子

- 输入: 基本块 B , 开始时 B 的所有非临时变量都是活跃的
- 输出: 各语句 i 上的变量的活跃性、后续使用信息
 - 在8之前的程序点上, i, j, a 仍然活跃; 且 j 在8上被使用
 - 在7之前的程序点上, $i, j, a, t4$ 活跃; 且 $t4$ 被7使用
 - 在6之前的程序点上, $i, j, a, t3$ 活跃, $t4$ 不活跃, $t3$ 被6使用
 - 在5之前的程序点上, $i, j, a, t2$ 活跃, $t3$ 不活跃
 - 在4之前的程序点上, ...

- 3) $t1 = 10 * i$
- 4) $t2 = t1 + j$
- 5) $t3 = 8 * t2$
- 6) $t4 = t3 - 88$
- 7) $a[t4] = 0.0$
- 8) $j = j + 1$



基本块和流图

■ 后续使用信息

- 可用于优化

■ 寄存器指派

如果当前某个变量存放于一个寄存器中, 且之后不会再被使用, 那么这个寄存器可被分派给别的变量

$a = b + c$
 $d = a + e$

LD R0, b
ADD R1, R0, c
ST a, R1

LD R1, a
ADD R2, R1, e
ST d, R2



LD R0, b // R0 = b
ADD R0, R0, c // R0 = R0 + c
ST a, R0 // a = R0

LD R0, a // R0 = a
ADD R0, R0, e // R0 = R0 + e
ST d, R0 // d = R0



基本块和流图

■ 确定基本块中的活跃性、后续使用

- 输入: 基本块 B , 开始时 B 的所有非临时变量都是活跃的
- 输出: 各语句 i 上的变量的活跃性、后续使用信息
- 方法:
 - 从 B 的最后一个语句开始反向扫描
 - 对于每个语句 $i: x = y + z$
 - 从 B 的最后一个语句开始反向扫描
 - 对于每个语句 $i: x = y + z$
 1. 令语句 i 和 x, y, z 的当前活跃性信息/使用信息关联
 2. 设置 x 为不活跃、无后续使用
 3. 设置 y 和 z 为活跃, 并指明它们的下一次使用为语句 i
 - 注意: 2和3的顺序



基本块和流图

■ 流图的构造

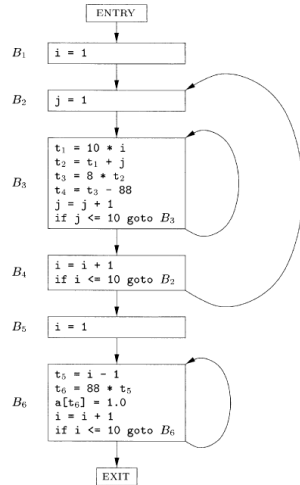
- 两个顶点 B 和 C 之间有一条有向边当且仅当基本块 C 的第一个指令可能在 B 的最后一个指令之后执行
 - 从 B 的结尾指令是一条跳转到 C 的开头的条件/无条件语句
 - 在原来的序列中, C 紧跟在 B 之后, 且 B 的结尾不是无条件跳转语句
- 称 B 是 C 的前驱, C 是 B 的后继
- 入口和出口结点
 - 流图中额外添加的边, 不与中间代码 (基本块) 对应
 - 入口到第一条指令有一条边
 - 从任何可能最后执行的基本块到出口有一条边



基本块和流图

■ 流图的构造

- 因跳转而生成的边
 - B3 → B3
 - B4 → B2
 - B6 → B6
- 因为顺序而生成的边
 - 其它



基本块和流图

■ 循环

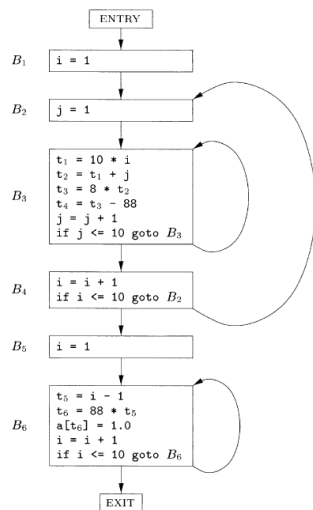
- 程序的大部分运行时间花费在循环上
 - 循环是识别的重点
- 循环的定义
 - 循环L是一个结点集合
 - 存在一个循环入口 (loop entry) 结点, 是唯一的、前驱可以在循环L之外的结点, 到达其余结点的路径必然先经过这个入口结点
 - 其余结点都存在到达入口结点的非空路径, 且路径都在L中



基本块和流图

■ 循环的例子

- 循环
 - {B3}
 - {B6}
 - {B2, B3, B4}
- 对于 {B2, B3, B4} 的解释
 - B2为入口结点
 - B1, B5, B6不在循环内的理由
 - 到达B1可不经B2
 - B5, B6没有到达B2的结点



基本块的优化

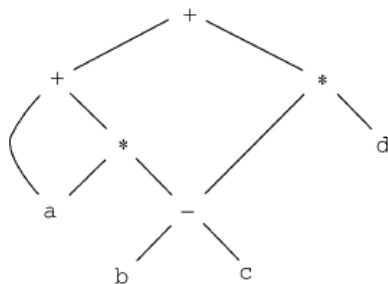
■ 针对基本块的局部优化

- 针对基本块的优化可以有很好的效果
- 基本块中各个指令要么都执行, 要么都不执行



基本块的优化

■ 回顾: 表达式的DAG



基本块的优化

■ 基本块可以用DAG表示

- 每个变量有对应的DAG的结点, 代表初始值
- 每个语句s有一个相关的结点N, 代表计算得到的值
 - N的子结点对应于 (其运算分量当前值的) 其它语句
 - 结点N的标号是s的运算符
 - N和一组变量关联, 表示s是在此基本块内最晚对它们定值的语句
- 输出结点: 结点对应的变量在基本块出口处活跃
- 从DAG, 我们可以知道各个变量最后的值和初始值的关系

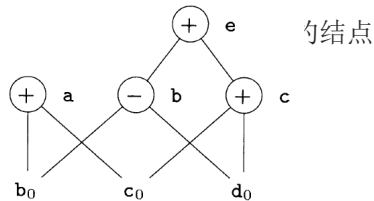


基本块的优化

■ 消除死代码

- 在DAG上消除没有附加活跃变量的根结点（没有父结点的结点），即消除死代码
- 如果图中c、e不是活跃变量

死代码=? 不可达代码



基本块的优化

■ 数组引用的DAG表示

- 从数组取值的运算 $x=a[i]$ 对应于 $=[]$ 的结点
 - x作为这个结点的标号之一
 - 该节点的左右子节点分别代表数组初始值和下标
- 对数组赋值（例如 $a[j]=y$ ）的运算对应于 $[]=$ 的结点，没有关联的变量、且杀死所有依赖于a的变量
 - 三个子节点分别表示 a_0 、j和y
- 数组赋值隐含的意思：将数组作为整体考虑，对数组元素赋值即改变整个数组



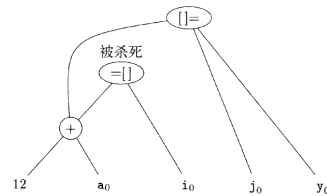
基本块的优化

■ 数组引用的DAG表示实例

- 设a是数组，b是指针

- $b=12+a$
- $x=b[i]$
- $b[j]=y$

- 注意a是被杀死的结点的孙结点
- 一个结点被杀死，意味着它不能被复用
 - 考虑再有指令 $m=b[i]$?



基本块的优化

■ 数组引用实例

- $x=a[i]$
 - $a[j]=y$
 - $z=a[i]$
- a[i]是公共子表达式吗?

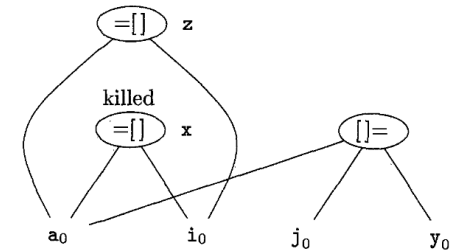
- 注意：a[j]可能改变a[i]的值，因此不能和普通的运算符一样构造相应的结点



基本块的优化

■ 数组引用的DAG表示实例

- 基本块
 - $x=a[i]$
 - $a[j]=y$
 - $z=a[i]$



基本块的优化

- 根据三地址指令序列生成机器指令，假设
 - 每个三地址指令只有一个对应的机器指令
 - 有一组寄存器用于计算基本块内部的值
- 主要目标
 - 尽量减少加载和保存指令，即最大限度利用寄存器
- 重点考虑寄存器的使用方法
 - 执行运算时，运算分量必须放在寄存器中
 - 用于临时变量
 - 存放全局的值
 - 进行运行时管理（比如：栈顶指针）



基本块的优化



- DAG 是如何指导指令的选择的？一个例子
- 我们定义一系列的规则，用于支持指令的选择和排列

Patterns	Instructions
$\odot_x$ a leaf node	LD R, x // R is a new register
$\oplus_x$ an operator node	ADD R _n , R _{o1} , R _{o2} ST x, R _n // R _n is a new register; // R _o are previous registers



基本块的优化



- $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



基本块的优化



- DAG 是如何指导指令的选择的？一个例子
- 我们定义一系列的规则，用于支持指令的选择和排列

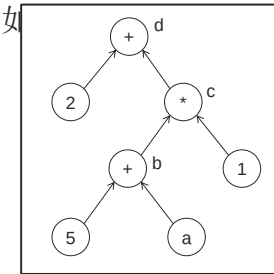
Patterns	Instructions
$\odot_x$ a leaf node	LD R, x // R is a new register
$\oplus_x$ an operator node	ADD R _n , R _{o1} , R _{o2} ST x, R _n // R _n is a new register; // R _o are previous registers



基本块的优化



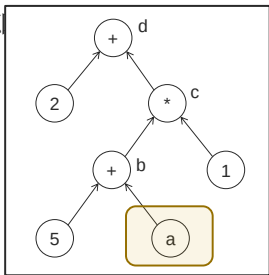
- $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



基本块的优化



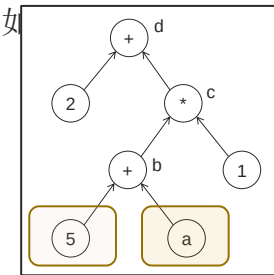
- $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



基本块的优化



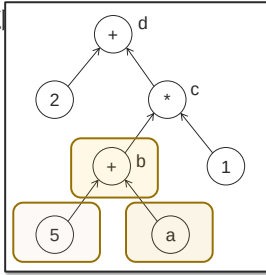
- $b = a + 5$
 $c = b * 1$
 $d = 2 + c$





基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



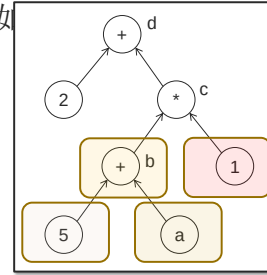
的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b	R2



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



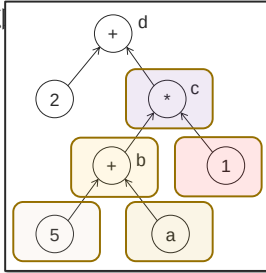
的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b	R2
LD	R3,	1



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



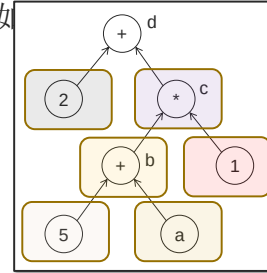
的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c	R4



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



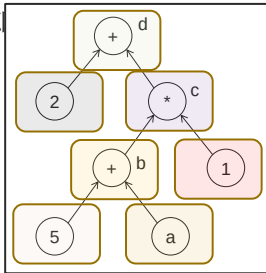
的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c	R4
LD	R5,	2



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



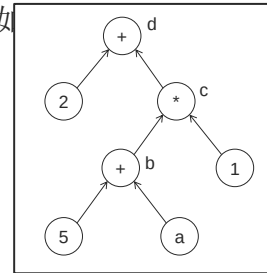
的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c	R4
LD	R5,	2
ADD	R6,	R4, R5
ST	d,	R6



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



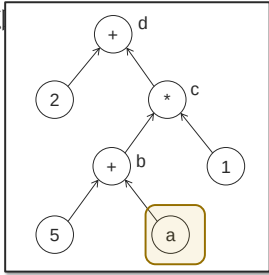
的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c	R4
LD	R5,	2
ADD	R6,	R4, R5
ST	d,	R6



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



的? 一个例子

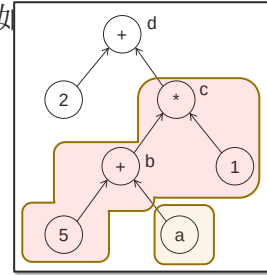
LD	R1,	5
ADD	R2,	R0, R1
ST	b,	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c,	R4
LD	R5,	2
ADD	R6,	R4, R5
ST	d,	R6

LD R0, a



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



的? 一个例子

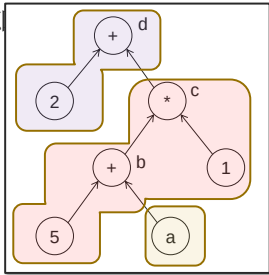
LD	R1,	5
ADD	R2,	R0, R1
ST	b,	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c,	R4
LD	R5,	2
ADD	R6,	R4, R5
ST	d,	R6

LD	R0,	a
ADD	R1,	R0, 5
ST	b,	R1
ST	c,	R1



基本块的优化

■ $b = a + 5$
 $c = b * 1$
 $d = 2 + c$



的? 一个例子

LD	R1,	5
ADD	R2,	R0, R1
ST	b,	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c,	R4
LD	R5,	2
ADD	R6,	R4, R5
ST	d,	R6

LD	R0,	a
ADD	R1,	R0, 5
ST	b,	R1
ST	c,	R1
ADD	R2,	R1, 2
ST	d,	R2



寄存器分配

实际情况

- $a = b + c$
 - LD R0, b
 - LD R1, c
 - ADD R2, R0, R1
 - ST a, R2
- 隐含的假设
 - 无限多寄存器
 - 不知道变量是否在寄存器中
 - 所有变量以后都有用

预先判断需要哪些加载指令把必须的运算分量加载进寄存器

有限的寄存器，加载时预判选用哪个寄存器

如果必要才把结果存入内存



寄存器分配

- 在一个基本块中的寄存器分配
- **MAXLIVE**: 每条指令处存活值的最大数量
 - 若 $\text{MAXLIVE} \leq k \rightarrow$ 巨简单（无需溢出）！
 - 若 $\text{MAXLIVE} > k \rightarrow$ 必须将部分值溢出到内存



寄存器分配

- 在一个基本块中的寄存器分配

```

1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8        // contents(R1) <- z * (x-y) - 1028 * y
  
```




寄存器分配



```
1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8      // contents(R1) <- z * (x-y) - 1028 * y
```



寄存器分配



```
1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8      // contents(R1) <- z * (x-y) - 1028 * y
```



寄存器分配



```
1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8      // contents(R1) <- z * (x-y) - 1028 * y
```



寄存器分配



```
1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8      // contents(R1) <- z * (x-y) - 1028 * y
```



寄存器分配



```
1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8      // contents(R1) <- z * (x-y) - 1028 * y
```



寄存器分配



```
1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1   R8      // contents(R1) <- z * (x-y) - 1028 * y
```



寄存器分配



```

1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1    R8      // contents(R1) <- z * (x-y) - 1028 * y
    
```



寄存器分配



```

1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1    R8      // contents(R1) <- z * (x-y) - 1028 * y
    
```



寄存器分配



```

1. LD    R1    #1028    // R1
2. LD    R2    *R1      // R1 R2
3. MUL   R3    R1    R2  // R1 R2 R3
4. LD    R4    x        // R1 R2 R3 R4
5. SUB   R5    R4    R2  // R1 R3 R5
6. LD    R6    z        // R1 R3 R5 R6
7. MUL   R7    R5    R6  // R1 R3 R7
8. SUB   R8    R7    R3  // R1 R8
9. ST    *R1    R8      //
    
```



寄存器分配



```

1. LD    R1    #1028    // R1 <- 1028
2. LD    R2    *R1      // R2 <- contents(R1), assume y
3. MUL   R3    R1    R2  // R3 <- 1028 * y
4. LD    R4    x        // R4 <- x
5. SUB   R5    R4    R2  // R5 <- x-y
6. LD    R6    z        // R6 <- z
7. MUL   R7    R5    R6  // R7 <- z * (x-y)
8. SUB   R8    R7    R3  // R8 <- z * (x-y) - 1028 * y
9. ST    *R1    R8      // contents(R1) <- z * (x-y) - 1028 * y
    
```



寄存器分配



```

1. LD    R1    #1028    // R1
2. LD    R2    *R1      // R1 R2
3. MUL   R3    R1    R2  // R1 R2 R3
4. LD    R4    x        // R1 R2 R3 R4
5. SUB   R5    R4    R2  // R1 R3 R5
6. LD    R6    z        // R1 R3 R5 R6
7. MUL   R7    R5    R6  // R1 R3 R7
8. SUB   R8    R7    R3  // R1 R8
9. ST    *R1    R8      //
    
```



寄存器分配



```

1. LD    R1    #1028    // R1
2. LD    R2    *R1      // R1 R2
3. MUL   R3    R1    R2  // R1 R2 R3
4. LD    R4    x        // R1 R2 R3 R4
5. SUB   R5    R4    R2  // R1 R3 R5
6. LD    R6    z        // R1 R3 R5 R6
7. MUL   R7    R5    R6  // R1 R3 R7
8. SUB   R8    R7    R3  // R1 R8
9. ST    *R1    R8      //
    
```

MAXLIVE = 4



寄存器分配

1. LD	R1	#1028		// R1
2. LD	R2	*R1		// R1 R2
3. MUL	R3	R1	R2	// R1 R2 R3
4. LD	R4	x		// R1 R2 R3 R4
5. SUB	R5	R4	R2	// R1 R3 R5
6. LD	R6	z		// R1 R3 R5 R6
7. MUL	R7	R5	R6	// R1 R3 R7
8. SUB	R8	R7	R3	// R1 R8
9. ST	*R1	R8		//

MAXLIVE = 4

4 个寄存器就够了! ?



寄存器分配

1. LD	R1	#1028		// R1
2. LD	R2	*R1		// R1 R2
3. MUL	R3	R1	R2	// R1 R2 R3
4. LD	R4	x		// R1 R2 R3 R4
5. SUB	R5	R4	R2	// R1 R3 R5
6. LD	R6	z		// R1 R3 R5 R6
7. MUL	R7	R5	R6	// R1 R3 R7
8. SUB	R8	R7	R3	// R1 R8
9. ST	*R1	R8		//

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1
2. LD	R2	*R1		// R1 R2
3. MUL	R3	R1	R2	// R1 R2 R3
4. LD	R4	x		// R1 R2 R3 R4
5. SUB	R5	R4	R2	// R1 R3 R5
6. LD	R6	z		// R1 R3 R5 R6
7. MUL	R7	R5	R6	// R1 R3 R7
8. SUB	R8	R7	R3	// R1 R8
9. ST	*R1	R8		//

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1
2. LD	R2	*R1		// R1 R2
3. MUL	R3	R1	R2	// R1 R2 R3
4. LD	R4	x		// R1 R2 R3 R4
5. SUB	R2	R4	R2	// R1 R3 R2
6. LD	R6	z		// R1 R3 R2 R6
7. MUL	R7	R2	R6	// R1 R3 R7
8. SUB	R8	R7	R3	// R1 R8
9. ST	*R1	R8		//

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1
2. LD	R2	*R1		// R1 R2
3. MUL	R3	R1	R2	// R1 R2 R3
4. LD	R4	x		// R1 R2 R3 R4
5. SUB	R2	R4	R2	// R1 R3 R2
6. LD	R6	z		// R1 R3 R2 R6
7. MUL	R7	R2	R6	// R1 R3 R7
8. SUB	R8	R7	R3	// R1 R8
9. ST	*R1	R8		//

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1
2. LD	R2	*R1		// R1 R2
3. MUL	R3	R1	R2	// R1 R2 R3
4. LD	R4	x		// R1 R2 R3 R4
5. SUB	R2	R4	R2	// R1 R3 R2
6. LD	R6	z		// R1 R3 R2 R6
7. MUL	R7	R2	R6	// R1 R3 R7
8. SUB	R8	R7	R3	// R1 R8
9. ST	*R1	R8		//

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R2	R4	R2	// R1 R3	R2
6. LD	R4	z		// R1 R3	R2 R4
7. MUL	R7	R2	R4	// R1 R3	R7
8. SUB	R8	R7	R3	// R1	R8
9. ST	*R1	R8		//	

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R2	R4	R2	// R1 R3	R2
6. LD	R4	z		// R1 R3	R2 R4
7. MUL	R7	R2	R4	// R1 R3	R7
8. SUB	R8	R7	R3	// R1	R8
9. ST	*R1	R8		//	

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R2	R4	R2	// R1 R3	R2
6. LD	R4	z		// R1 R3	R2 R4
7. MUL	R2	R2	R4	// R1 R3	R2
8. SUB	R8	R2	R3	// R1	R8
9. ST	*R1	R8		//	

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R2	R4	R2	// R1 R3	R2
6. LD	R4	z		// R1 R3	R2 R4
7. MUL	R2	R2	R4	// R1 R3	R2
8. SUB	R8	R2	R3	// R1	R8
9. ST	*R1	R8		//	

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R2	R4	R2	// R1 R3	R2
6. LD	R4	z		// R1 R3	R2 R4
7. MUL	R2	R2	R4	// R1 R3	R2
8. SUB	R2	R2	R3	// R1	R2
9. ST	*R1	R2		//	

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R5	R4	R2	// R1 R3	R5
6. LD	R6	z		// R1 R3	R5 R6
7. MUL	R7	R5	R6	// R1 R3	R7
8. SUB	R8	R7	R3	// R1	R8
9. ST	*R1	R8		//	

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$



寄存器分配

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R5	R4 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$



寄存器分配

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R5	R4 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$



寄存器分配

1. 在一个基本块中的寄存器分配

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R5	R4 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$



寄存器分配

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R5	R4 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$



寄存器分配

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R5	R4 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$

重新加载 R3, LD R3 v



寄存器分配

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R3	x	// R1 R2 R3
5. SUB	R5	R3 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

if $k < 4$, e.g., $k = 3$

重新加载 R3, LD R3 v



寄存器分配

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	需要考虑将 R3 中的内容进行转存, ST v R3
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R5	R4	R2	// R1 R3 R5	MAXLIVE = 4
6. LD	R6	z		// R1 R3 R5 R6	if k < 4, e.g., k = 3
7. MUL	R7	R5	R6	// R1 R3 R7	
8. SUB	R8	R7	R3	// R1 R8	
9. ST	*R1	R8		//	



寄存器分配

- 记录各个值对应的位置
 - **寄存器**描述符: 跟踪各个寄存器都存放了哪些变量的当前值
 - **地址**描述符: 某个变量的当前值存放在哪个或哪些位置(包括内存位置和寄存器)



寄存器分配

- 运算语句: $x = y + z$
 - 调用 `getReg(x=y+z)`, 为 x, y, z 选择寄存器 R_x, R_y, R_z
 - 查 R_y 的寄存器描述符, 如果 y 不在 R_y 中则生成指令
 - `LD R_y y'` (y' 表示存放 y 值的当前位置)
 - 类似地, 确定是否生成 `LD R_z, z'`
 - 生成指令 `ADD R_x, R_y, R_z`
- 赋值语句: $x = y$
 - `getReg(x=y)` 总是为 x 和 y 选择相同的寄存器
 - 如果 y 不在 R_y 中, 生成机器指令 `LD R_y, y`
- 基本块的收尾
 - 如果变量 x 在出口处活跃, 且 x 现在不在内存, 那么生成指令 `ST x, R_x`



寄存器分配

- 依次考虑各三地址指令, 尽可能把值保留在寄存器中, 以减少寄存器/内存之间的数据交换
- 为一个三地址指令生成机器指令时
 - 只有当运算分量不在寄存器中时, 才从内存载入
 - 尽量保证只有当寄存器中的值不被使用时, 才把它覆盖掉



寄存器分配

- 代码生成算法框架
 - 遍历三地址指令
 1. 使用 `getReg()` 函数选择寄存器
 2. 生成指令
- 重要子函数: `getReg(I)`
 - 根据寄存器描述符和地址描述符、数据流信息, 为三地址指令 I 选择最佳的寄存器
 - 得到的机器指令的质量依赖于 `getReg` 函数选取寄存器的算法



寄存器分配

- 代码生成同时更新寄存器和地址描述符
- 处理普通指令时生成 `LD R x`
 - R 的寄存器描述符: 只包含 x
 - x 的地址描述符: R 作为新位置加入到 x 的位置集合中
 - 从任何不同于 x 的变量的地址描述符中删除 R
- `ST x, R`
 - 修改 x 的地址描述符, 包含自己的内存位置



寄存器分配

- ADD Rx, Ry, Rz
 - Rx的寄存器描述符只包含x
 - x的地址描述符只包含Rx（不包含x的内存位置！）
 - 从任何不同于x的变量的地址描述符中删除Rx。
- 处理x=y时，
 - 如果生成LD Ry y，按照第一个规则处理；
 - 把x加入到Ry的寄存器描述符中（Ry存放了x和y的当前值）；
 - 修改x的地址描述符，使它只包含Ry



寄存器分配

- a、b、c、d在出口处活跃
- t、u、v是局部临时变量

```

t = a - b
u = a - c
v = t + u
a = d
d = v + u

```

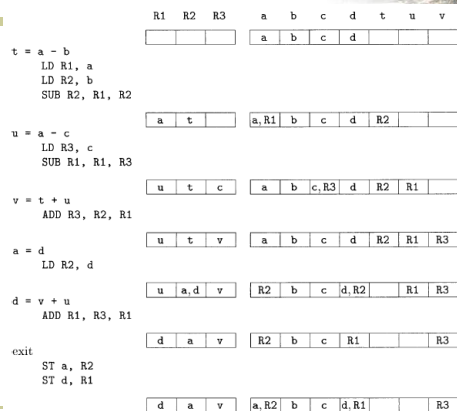


寄存器分配

```

t = a - b
u = a - c
v = t + u
a = d
d = v + u

```



生成的指令以及寄存器和地址描述符的改变过程



寄存器分配

- getReg的目标：减少LD/ST指令
- 为运算分量y分配寄存器（如果不在寄存器中，且没有空闲寄存器）
 - 寻找一个寄存器R，假设已经有某个变量v在R中了（R的寄存器描述符中包含了v）
 - 如果v的地址描述符表明还可以在别的地方找到v，DONE
 - v就是x（即结果），且x不是运算分量之一，DONE
 - 如果v在此之后不会被使用，DONE
 - 生成保存指令ST v R（溢出操作）并修改v的地址描述符，如果R中存放了多个变量的值，那么需要生成多条ST指令（选尽可能少ST的）；

- 关键：借用了v所在的寄存器，不能随意把v冲掉



寄存器分配

- getReg的目标：减少LD/ST指令
- 任务：为运算分量和结果分配寄存器
- 为 $x = y + z$ 指令选择寄存器
- 为运算分量y分配寄存器
 - 如果已经在某个寄存器中，不需要进行处理，选择这个寄存器
 - 如果不在寄存器中，且有空闲寄存器，选择一个空闲寄存器
 - 如果不在寄存器中，且没有空闲寄存器？



寄存器分配

- 为x选择寄存器Rx（基本方法和把y从内存LD时一样）
- 区别
 - 只存放x的值的寄存器总是可接受的，因为x计算出了一个新的值
 - 如果y在指令I之后不再使用，且Ry仅仅保存了y的值，那么Ry同时也可以作为Rx
- 处理x=y时，我们总是让Rx=Ry



寄存器分配

- 看一个示例习题： $d := (a-b)+(a-c)+(a-c)$ 的三地址代码序列：



寄存器分配

- 看一个示例习题： $d := (a-b)+(a-c)+(a-c)$ 的三地址代码序列：

$t_1 := a - b$
 $t_2 := a - c$
 $t_3 := t_1 + t_2$
 $d := t_3 + t_2$

假设只有两个寄存器，做寄存器分配？



寄存器分配

statements	code generated	register descriptor	address descriptor
$t_1 := a - b$	LD R0,a LD R1,b SUB R1,R0,R1	R0含a R1含 t_1	a in R0 t_1 in R1
$t_2 := a - c$	ST t1,R1 LD R1,c SUB R0,R0,R1	R1含c R0含 t_2	c in R1 t_2 in R0
$t_3 := t_1 + t_2$	LD R1,t1 ADD R1,R1,R0	R1含 t_3	t_3 in R1 t_2 in R0
$d := t_3 + t_2$	ADD R1,R1,R0	R1含d R0含 t_2	d in R1 t_2 in R0
	ST d,R1		d在R1和内存中



(1) $t = a * b$ (指令1分，描述符1分)

LD R1,a

LD R2,b

MUL R1,R1,R2

R1	R2	x	a	b	t	u
T	b	x	a	b,R2	R1	u

(2) $u = x + t$ (指令2分)

LD R2,x

ADD R2 R1 R2

R1	R2	x	a	b	t	u
T	u	x	a	b	R1	R2

(3) $x = u$ (描述符3分)

R1	R2	x	a	b	t	u
T	u,x	R2	a	b	R1	R2

(4) (指令3分)

ST t,R1

ST u,R2

ST x,R2

R1	R2	x	a	b	t	u
t	u,x	x,R2	a	b	t,R1	u,R2



八、简答题（共1题，共10分）

假设有两个可用的寄存器 R1 和 R2，使用简单代码生成算法，

为如下三地址代码生成机器代码。

$t = a * b$

$u = x + t$

$x = u$

- 给出每条三地址代码生成机器码后的寄存器描述符和内存描述符（请在表格左边空白部分写出三地址代码，并在表格中补充写入对应的描述符内容）。

- 结束时将寄存器中的变量写回内存中。

寄存器代码格式：

取数指令：LD R, var // 将内存空间中变量 var 的值存入寄存器 R 中。

存数指令：ST var, R // 将寄存器 R 中的值存入变量 var 的内存空间中。

双目运算指令：OP dest, src1, src2 // 使用 OP 对寄存器 src1 和 src2 中的值进行计算，结果存入寄存器 dest 中（OP 包括 ADD、SUB、MUL、DIV）。

初始状态：

R1	R2	x	a	b	t	u
		x	a	b	t	u

- $t = a * b$

R1	R2	x	a	b	t	u
		x	a	b	R1	u

- $u = x + t$

R1	R2	x	a	b	t	u
t	u	x	a	b	R1	R2

- $x = u$

R1	R2	x	a	b	t	u
t		x	a	b	R1	

-

R1	R2	x	a	b	t	u
t	u,x	x,R2	a	b	t,R1	u,R2